

Detection and Analysis of DDoS Attack using a Collaborative Network Monitoring Stack

1st Muhammad Aashiq Moosa

Department of Computing
National College of Ireland
Dublin, Ireland
maashiq13@gmail.com

1st Apurva K. Vangujar

School of Computer Science & IT
University College Cork
Cork, Ireland
a.vangujar@cs.ucc.ie

2nd Dnyanesh Pramod Mahajan

Department of Computing
National College of Ireland
Dublin, Ireland
dnyaneshmahajan12@gmail.com

Abstract—In recent years, Distributed Denial of Service (DDoS) attacks have become common and have evolved into a standard method of targeting entities. These attacks can be used to make money or to damage the network and systems of an organisation. The majority of recent research in this field has focused on implementing various machine learning (ML) strategies to detect and thwart these attacks. Even though these techniques have a high rate of accuracy, they occasionally struggle to detect zero-day vulnerabilities or manage intense network traffic. This investigation shifts its focus from DDoS detection to the creation of a cost-effective network monitoring system. The intended solution will provide real-time visibility into a simulated network environment on Graphical Network Simulator-3 (GNS3) and incorporate a machine learning (ML) engine for enhanced DDoS detection via network monitoring tools such as *Prometheus*, *Zabbix*, and *Grafana*. This initiative seeks to provide organisations with a comprehensive, cost-effective solution that extends beyond DDoS mitigation by emphasising a more cost-effective and efficient monitoring system. During the implementation of the project, a few technical obstacles prevented the integration of certain components into the solution, as will be described in the paper. However, an integrated solution and collaborative design will increase the solution's ability to detect more sophisticated DDoS attacks.

Index Terms—DDoS Attack, Collaborative Network, Monitoring Tools, Prometheus, Zabbix, Grafana

I. INTRODUCTION

DDoS is a cyberattack that aims to prevent authorised users from accessing particular services by overwhelming system resources, network capacity, or other vital components. This type of attack can affect a variety of network components, such as servers, databases, end-user devices, and cloud platforms. DDoS attacks can be difficult to identify due to the fact that malicious traffic often resembles ordinary user activity. DDoS attacks pose a substantial threat to service accessibility because they aim to disrupt legitimate user-server interactions [1].

According to [2], there were 74% more DDoS attacks in 2022 compared to the previous year. Fintech was particularly susceptible, accounting for 34% of these cases. Moreover, such attacks have increased by a factor of twelve in financial services. DDoS attacks are among the most dangerous online hazards, per Norton. These attacks can occur without warning,

impacting any aspect of a website's functionality or assets, and resulting in significant downtime and monetary losses. DDoS attacks became more prevalent in 2022 than in previous years. Both the frequency and duration of assaults have increased in recent years stretches. During the second quarter of 2021, a typical DDoS attack will last approximately 30 minutes, according to [3].

DDoS attacks pose a significant threat to networks and businesses. It has the potential to substantially disrupt business operations and services, leading to financial losses. The increasing frequency of DDoS attacks indicates that assailants employ them for a variety of objectives. Urgently required is a low-cost solution or method for detecting these types of attacks in real time.

Monitoring a network is necessary for network management operations and is used for a variety of important tasks. The early detection of patterns and trends in network traffic and devices is a crucial function of network monitoring. These observations allow network administrators to comprehend the current state of a network and then modify it to enhance the observed state [4]. This paper presents monitoring tools that are designed to easily monitor network activity and identify patterns in network traffic.

A. Related Work

1) *DDoS Attack Detection*: Existing security methods [5] either provide insufficient protection against these attacks or are only effective against a subset of DDoS attacks. Understanding the principal components of a DDoS attack is crucial, as they can abruptly alter the used port, protocol, or operation mode. Methods for threat detection using Machine Learning (ML) have been extensively investigated. However, it is unclear which characteristics are essential and which methods are the most effective for detecting attacks. The three most common defensive techniques are detection, filtering, and traceback. It can be difficult to distinguish between legitimate and malicious traffic with precision. Filtering may slow down the network when there is a lot of traffic, and traceback only functions when there is little traffic. The preponderance of currently employed detection methods have lower success rates. When an attack employs legitimate requests, it can be difficult to distinguish attack traffic from normal traffic [6].

Due to the enormous quantity of data required for analysis, real-time network detection can be challenging. Statistical techniques are effective at detecting atypical resource utilisation patterns resulting from DDoS attacks. The inability of statistical analysis to recognise the typical distribution of network packets, necessitating its approximation as a constant distribution, is a disadvantage of using statistical analysis for detection. Although data-mining-based ML systems have proven to be highly accurate at detecting DDoS attacks, they are not without their own challenges. Their lengthy learning period is a significant disadvantage that precludes their use in real-time operations at present.

A paper published in 2012 by Parevz *et al.*, proposed a behavior model to combat malware and detect Windows Portable Executable (PE) files. The model analyses the Application Programming Interface strings made by the files. The model in testing has an accuracy rate of 98.6% cite-faruki2012behavioural.

In 2013, Parevz *et al.*, publish a paper, presenting AndroSimilar, a method for detecting malicious Android applications in smartphones. The motivation behind this paper is the growing popularity of smartphone technology which in turn increases the malwares threats in smartphones [7].

A paper published by Parevz *et al.* in 2015 presents an automated application vetting and malware analysis platform that combines static and dynamic analysis in order to enhance analysis accuracy and efficiency named DroidAnalyst [8].

A survey published in 2015, reviewed the advantages and disadvantages of security mechanisms and methodologies in Android security and smartphones security. The survey concludes by recommending a hybrid approach for android malware analysis using static and dynamic analysis. [9]

In 2017, Snort and Suricata, two open-source Intrusion Detection Systems (IDS), were evaluated to determine how well they could identify malicious traffic on computer networks. The effectiveness of both IDSs was determined at a network speed of 10 Gbps. Shah *et al.* [10] discovered that Suricata required more computational resources than Snort, but handled network traffic faster and with fewer packet drops. Snort was selected for additional testing due to its superior detection precision. The study found, however, that Snort generated a significant number of false positive alerts. According to Ye *et al.* [11], neural network algorithms cannot be utilised in 2018. Using Mininet and Floodlight, they simulated an SDN environment and devised a DDoS attack model using SVM classification techniques. During testing, they discovered that this method had an accuracy rate of 95.24% percent.

In 2019, Chaabouni *et al.* [12] published a paper describing the security of IoT devices are a critical concern. They carried out a survey of current IoT defense strategies. Its main focus is on network intrusion detection systems (NIDS). The research concluded that NIDS, with an eye on edge and fog computing methods, should be investigated in order to develop system architectures with lower resource consumption.

In 2021, Sun *et al.* [13] describes a method for utilising raw data to visually represent a variety of network traffic

characteristics. For this, the raw data was first divided into manageable segments. Then, a two-week's worth of network traffic data was used to train a model using a supervised neural network and a labelling technique based on expert knowledge. The first week's data served as the training set, while the second week's data served as the validation set. The validation results indicated precision values of 0.800 and 0.815 for identifying malicious Server Message Block (SMB) and TCP Synchronize Sequence Number (SYN) flooding, and precision scores of 0.980 for detecting ARP flooding. Another published study demonstrated that Python open-source code could be developed for IDS. Using ML models, the programme can detect all forms of cyberattacks, ensuring the security of modern networks. To address cybersecurity concerns, this technique by Yang *et al.* [14] can be simply created and replicated on additional intrusion detection datasets.

To identify DDoS attacks using the Naive Bayes method described in [15]. The dataset's characteristics were utilised to develop a prediction model. The optimal features were src ip, dst ip, flow dur, flow iat max, fwd iat max, and bwd iat tot. These results indicate that the accuracy of detecting DDoS attacks with the Naive Bayes method increased from 65.6% without feature selection to 69.6% with feature selection. Even though the Naive Bayes technique can differentiate between DDoS and modest attacks, its overall accuracy is low.

In [16], a novel detection mechanism designated Scattered Denial-of-Service Mitigation Tree Architecture (SDMTA) is introduced, offering an innovative approach to mitigating DDoS attacks in hybrid cloud settings. This proposed architecture places a strong emphasis on network monitoring to expedite the detection processes. Using a specific dataset, the SDMTA outperformed current state-of-the-art models in a number of important metrics. It obtained accuracy, specificity, and sensitivity rates of 99.7%, 98.32%, and 99.92%, respectively, eclipsing leading models at the time.

2) *Network Monitoring:* In Mohammed *et al.* [17], a network monitoring solution known as NetGraf is introduced. The purpose of this paper is to introduce NetGraf, an innovative monitoring solution that consolidates multiple data sources into a single interface. In addition, it provides ML libraries for anomaly discovery and data analysis in real-time. NetGraf employs a database backend to track performance trends and alert users when network performance declines. A monitoring system that employs end-to-end learning to aggregate multiple metrics and present them on a single dashboard can be very useful for network operations because it makes the data simple to examine. Network administrators may find it useful to employ the system's ML algorithms to monitor a network and generate alerts when anomalies are detected.

In 2022, Maasaoui [18] developed and evaluated an NSTAP (real-time network security traffic analysis platform). NSTAP examines and analyses traffic data to identify and block malicious traffic. Using a variety of visualization techniques, including charts, tables, and histograms, we demonstrate that the platform is capable of producing insightful and valuable

insights from big data sets using basic time-domain analytics. This research lays the groundwork for forthcoming ML-based automation solutions.

According to the current trend in identifying DDoS attacks on networks, the majority of research is devoted to evaluating the utility and precision of various ML algorithms for detecting DDoS attacks using provided datasets. Although little research has been conducted on evaluating the efficacy of the models on a real-time network. The project proposes developing a collaborative network monitoring solution that incorporates an IDS employing ML methods, monitoring a simulated network that would resemble an actual network, and evaluating the efficacy of this solution. This research could contribute to the creation of more effective strategies for defending against DDoS attacks, a growing hazard to network security.

B. Motivation

Traditional DDoS detection techniques rely heavily on traffic analysis to identify anomalous activity and rate limiting to mitigate excessive traffic from specific IP addresses. Challenge-response tests, such as CAPTCHAs, distinguish between genuine users and bots, whereas blacklisting targets known malicious IP addresses. By examining traffic patterns and packet content, respectively, Anomaly-based detection and Deep Packet Inspection (DPI) further improve detection. Cloud-based solutions disperse DDoS traffic throughout extensive networks. However, there are still gaps: false positives in traffic analysis, the potential blocking of legitimate users through rate limiting, the reactive nature of blacklisting, and the high cost of advanced solutions. The fast evolving nature of DDoS attacks underscores the need for more adaptive and affordable detection methods. There is an urgent need for a cost-effective solution or technique capable of detecting multiple types of attacks in real-time. This research is motivated primarily by the desire to implement solutions that are cost-effective for small and medium-sized companies.

C. Contribution

A collaborative network monitoring solution that monitors and identifies DDoS attacks on a real-time network is the proposed goal of this research. A single dashboard will provide an overview of the real-time network with this solution. This can help network monitoring teams detect and respond in real time to security threats.

The research aims to:

- This research creates a novel solution for collaborative network monitoring that combines various tools and techniques. Utilising the assets of multiple monitoring components, it provides a unified view of complex network activities.
- By integrating sophisticated analytics and visualisation in a seamless manner, the proposed solution significantly improves the ability to detect DDoS attacks in real time. It implements a proactive defence against these disruptive attacks.

- This research substantially advances network monitoring techniques and fortifies network infrastructures against emerging threats by enabling real-time DDoS detection, clarifying tool usage, and providing a holistic security evaluation.

D. Organization

The structure of the paper is as follows. Section II describes the techniques we utilised to conduct this research. The execution of this project is described in the implementation Section III. Section III-D provides visuals of the results of our investigations, and Section IV provides the conclusion, followed by future research. Lastly, the Appendix provides the configuration for our network.

II. OUR ARCHITECTURE

The implementation of the following methodology is adopted for this study. We are evaluating numerous data surveillance and analysis tools; after analysing their natures of work, we eliminated tools whose natures of work overlapped and redesigned our implementation strategy.

- 1) **Network Module** This module includes the installation of the following network infrastructure monitoring tools. Open-source tools can be deployed as listening agents on each node to monitor the specific metrics and statistics that each node is intended to collect.
 - a) *Prometheus* is an open-source monitoring system with a dimensional data model, a flexible query language, an efficient time-series database, and a contemporary approach to alerting.
 - b) *ntopng* network usage is monitored by a network traffic probe. Based on libpcap and it has been written in a portable fashion to virtually run on every Unix platform, macOS, and on Windows as well. It is utilised for traffic network monitoring.
 - c) *Zabbix* is an open-source monitoring and management utility that collects real-time metrics such as CPU utilisation, disk activity, bandwidth utilisation, and website visits.
 - d) *Grafana* is a cross-platform interactive analytics and visualisation web application. It offers web-based charts, graphs, and notifications when connected to supported data sources. It is an open-source solution for database analytics and monitoring.
- 2) **Data Collection** This method ultimately produces a data set containing the collected data that can be used to train a ML model. Even though it may appear simple at first glimpse, data collection is the first and most important step in the ML pipeline. It is a crucial component of the challenging data processing phase of the ML life cycle [19]. This module collects and aggregates data from monitoring tools. Collecting CPU, memory, disc storage, network interface, throughput, loss, and latency data from *Prometheus* and *Zabbix*, as well as flow data from *ntopng*. We gathered information covering all

features and parameters. In our data acquisition process, we opted for a real-time database strategy, choosing InfluxDB for its time-series data storage optimization.

- 3) **Data Processing** Often, the data acquisition phase is the most time-consuming and crucial phase of a Data Science project. Even if your solution is sophisticated or complex, if the source data is of low quality, the result will be negative. Consequently, it is crucial to ensure that all pertinent data for the intended analytical technique are accessible, accurate, and in the correct format. Various duties must be performed at this stage, depending on the specific condition being treated. Developing a method to generate or capture data that is presently unavailable is one such endeavour.
- 4) **Data Modelling** The majority of effort at this stage is devoted to selecting the optimal techniques, ensuring that the data are in the correct format for those methods, and then training the model. It is recommended at this stage to utilise multiple models so that they can be compared and contrasted during the review phase. This is significant because in Data Science projects, the ideal technique may not be what is anticipated, and understanding why is crucial.
- 5) **Solution Implementation** The proposed solution will employ an architecture comparable to that described in [17]. The configuration would consist of network elements that supervise the simulated network and direct the obtained metrics to a centralised database. The ML-based detection component of the solution is trained and evaluated using the accumulated data. The metrics from the monitoring components and the analytical results are then displayed on a centralised interface. This provides users with a comprehensive visual understanding of network activity.

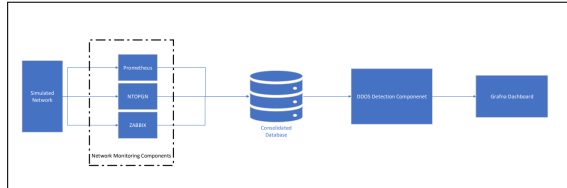


Fig. 1. Solution Architecture

- 6) **Performance Evaluation** This phase evaluates the collaborative network monitoring solution's ability to detect DDoS attacks. This includes evaluating the overall value of the system in detecting DDoS attacks, as well as the model's precision, false positive and false negative rates.
- 7) **Visualization Modelling** We meticulously examined the metrics collected by each monitoring tool and then refined our selection to include network-related metrics. On the "Grafana" interface, these selected metrics were then presented visually. We placed an emphasis on networking-specific metrics, including throughput, bandwidth, packet loss, round trip time, traffic loss, utilisation, network availability, flows, and network in-

terfaces, among others. This meticulous curation process allowed us to construct an optimised dashboard containing key networking community-relevant metrics. In addition, the *Grafana* dashboard displays baseline results and provides a comprehensive overview. Based on the data gathered and analysed by the ML component, the dashboard would also identify any anomalies in real-time. This procedure contributes to the creation of a comprehensive dashboard comprising valuable metrics that meet the requirements of users and network engineers. We facilitate seamless accessibility by combining multiple data sources into a single, user-friendly dashboard. Afterwards, we integrate the baseline learning monitoring system with ML libraries to analyse the data and promptly identify anomalies in real-time.

A. Research Resources

The solution architecture was influenced by the study cited in [17]. On the network software emulator GNS3, a real-time network will be developed and simulated to generate real-time network data. The monitoring component of the solution will monitor the data over time in order to provide enough data to train and evaluate the ML model and increase its accuracy and precision. Figure 2 depicts the simulated network that was designed and built using the GNS3 simulator.

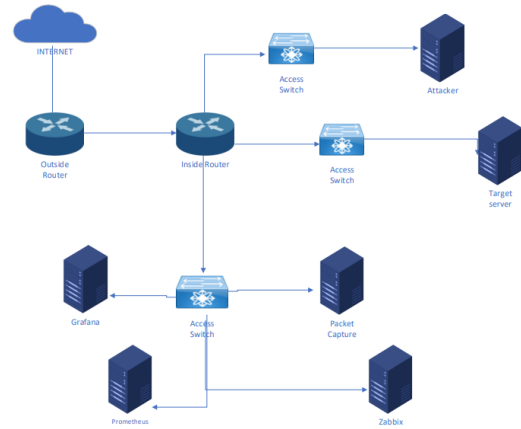


Fig. 2. Network Diagram for Network Simulation

B. Evaluation

As indicators of the model's efficiency, true positive rate, false positive rate, precision, and recall would be employed as performance metrics. The approach's ability to recognise and detect these attacks will be evaluated by simulating a variety of DDoS attack types on a simulated network using a variety of DDoS attack tools.

C. Design Specification

On a single physical system, a laptop, the virtual simulated network and architecture are designed and created. The following describes the machine's configuration:

TABLE I
SPECIFICATION OF STACK COMPONENTS

SN	Software Tools	Operating System	RAM	Storage	CPU
1	<i>Grafana</i>	Ubuntu 22.04	1 GB	10 GB	1V
2	<i>Prometheus</i>	Ubuntu 22.04	1 GB	10 GB	1V
3	<i>Zabbix</i>	Centos 7	2 GB	20 GB	1V
4	Target Web Server	Ubuntu 22.04	1 GB	10 GB	1V
5	Attacker Machine	Ubuntu 22.04	1 GB	10 GB	1V

- Performance Oriented CPU: Intel(R) Core(TM) i7-1065G7 CPU
- RAM: 16 GB DDR4
- Storage: 500 GB SSD Storage

GNS3 is used to simulate the network, and the monitoring stack is deployed on the GNS3 dynamics components and VMs. The stack component specifications are listed in Table I.

III. IMPLEMENTATION

This section describes the implementation specifications in detail. For detailed step-by-step configurations, please refer to the corresponding information provided in the Appendix.

A. Initial Network Setup

Initial configuration includes configuring the network to monitor and generate traffic. The network is composed of three internal network elements through which routers route traffic between nodes. The network is segmented as follows:

- 1) For the monitoring solution, the IP 192.168.10.0/24 is assigned to the monitoring system.
- 2) The attacker computer has the IP 192.168.20.0/24 for simulating the DDoS attack.
- 3) The target computer has the IP address 192.168.30.0/24 for hosting the DDoS web server.

As shown in Fig. 3, the routers are also connected to a nat cloud to provide external internet access to the nodes.

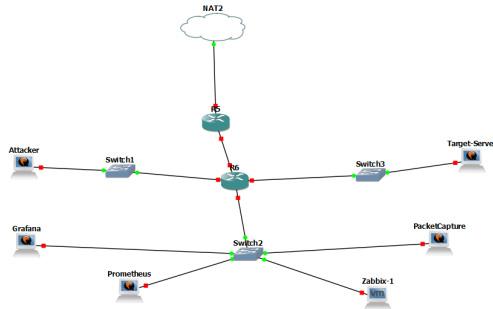


Fig. 3. Network implemented for Simulation in GNS3

According to the network diagram in Fig 3, the 'Outside Router' is the first router that connects the internal network to the external internet. The second router dubbed the 'Inside Router' provides network connectivity between the internal

nodes of the network. The internal router serves as the principal gateway for internal networks and their nodes.

B. Monitoring Solution

As described in Section II, our monitoring strategy includes well-known instruments like *Prometheus*, *Zabbix*, and *Grafana*. Initially, we investigated the use of *ntopng*, but as described in Section III-E, we ran into numerous obstacles related to storing metrics in InfluxDB.

The IP address 192.168.10.10/24 is assigned to the centos VM hosting *Zabbix*. The tool monitors internal nodes for various metrics using SNMPv2 for networking devices and the *Zabbix* agent for Ubuntu virtual machines. The metrics are then interrogated by *Grafana* to generate a dashboard for monitoring and visualising the network's health.

The IP address for *Prometheus* in our network is 192.168.10.155/24, which is assigned to an Ubuntu virtual machine. Using its "node exporter" and "snmp exporter" modules, *Prometheus* is capable of scraping the network for a variety of network metrics.

Additionally, *Grafana* is hosted on an Ubuntu VM with the IP address 192.168.10.13/24. The data source extensions for the *Zabbix* and *Prometheus* tools are then added in *Grafana* which is then used to query the data from the respective tools for visualisation. Since the focus of this study was DDoS attacks, the metrics acquired were network-related, such as the number of transmitted and received packets. The dashboard panels were designed to display unusual network or performance surges that could indicate a DDoS attack.

C. DDoS Detection Logic

This research employs a logic that compares the network passed through the network to a threshold. If the network exceeds the threshold, a suspected DDoS attack is occurring, and further investigation through the dashboard in *Grafana* can lead to the attack's source and target. Using a process of experimentation and refinement, we arrived at an initial threshold value of 80 after considering the specifics of our network environment. This particular threshold has proven effective in detecting DDoS attacks within our existing network architecture. However, it is essential to recognise that threshold values may vary depending on the configuration of individual networks.

Currently, the system searches for peaks in the network flow, such as incoming traffic volume or traffic patterns that deviate from normal behaviour, as well as peaks in the system health metrics, such as CPU usage and Disc usage. The surges were established by manually monitoring the network for a period of time, allowing the simulated network to operate normally and establishing the network's normal behaviour. As indicated previously, the solution's traffic analyzer component was not integrated due to technical limitations. The idea behind integrating a traffic analyzer component would then allows us to visualise traffic insights a packet data on *Grafana* which would then in turn allows us to use packet capture metrics to detect sophisticated DDoS attacks like ICMP flood attack

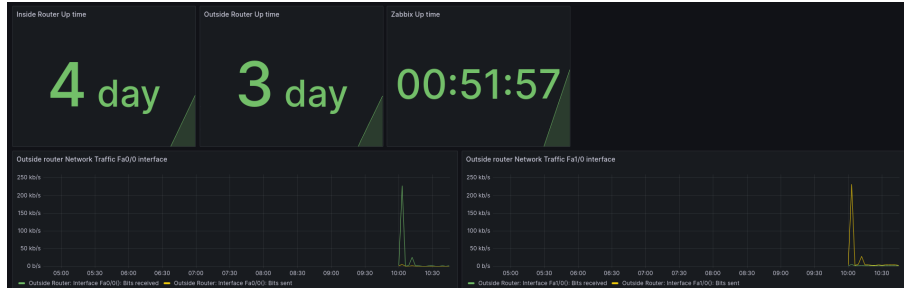


Fig. 4. Server Host Metrics

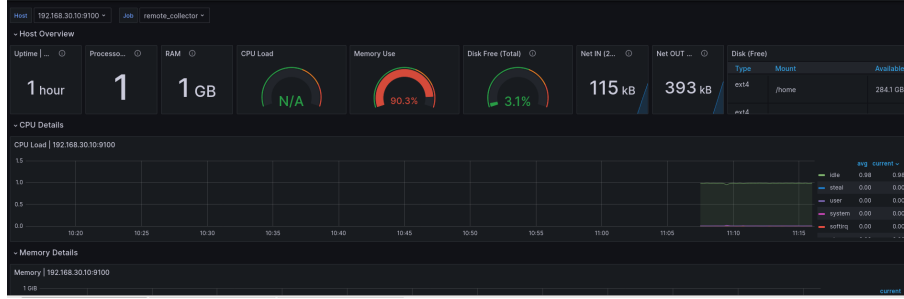


Fig. 5. Network Metrics

based on the volume of ping packets sent or TCP SYN flood attack based on the number of TCP connections that are open.

D. Solution View

As previously stated, *Grafana* serves as the front-end interface, providing a unified visualisation of the network via dashboards with a single panel and a single view. These displays are comprised of a variety of panels derived from the integrated modules, and they monitor the solution effectively. Illustrated in Figures 8 and 9, we showcase the visual outputs obtained through *Grafana*.

Figure 8 depicts the server host metrics of one of the monitored nodes (Target Web Server). The dashboard displays node metrics such as CPU utilisation, network bandwidth, and disc usage. A performance surge in these graphs may indicate DDoS activity. Figure 9 illustrates the server's network overview of the simulated infrastructure, including network traffic, system availability, etc. Set threshold values for monitoring and detecting DDoS activity. If the threshold is exceeded, the panel of the dashboard would turn red to indicate a surge, alerting the user of suspicious activity.

By displaying critical indicators, configuring intelligent alerts, and enabling rapid drill-downs, organisations can stay one step ahead of intruders. This ensures service continuity and robust network security at an economical cost.

E. Challenges

Implementing a packet analysis tool that could provide insight into the type of network traffic transmitted within the network was part of the initial implementation plan. However, during the duration of the undertaking, a number of technical

obstacles arose. In addition to *ntopng*, *zeek* or *tshark*, these utilities were considered for implementation.

Grafana though having plenty of modules to integrate with data sources, unfortunately the tool is limited and does not have integration connectors or API connector to the mentioned tools. This necessitated the use of a "middle man" in order for the packet analysis tool and *Grafana* to communicate. Initially, the time series database *influxdb* was used. However, the packet analysis instrument was incapable of storing its data in *influxdb*. Next, an attempt was made to use *elasticsearch* to index the time series data so that it could be sent to *Grafana* for visualisation. Despite the fact that the tool stored the packet captures in indexes to *elasticsearch*, *textitGrafana* was unable to query the data, preventing the packet capture from being visualised in the Dashboard.

Due to these obstacles, the packet analysis instrument was omitted and saved for future study. However, a collaborative tool that provides visualisation and dashboards for the overall health of the network as well as insights on the network's traffic under one "view" for a User or an Administrator could prove to be an effective solution offering.

IV. CONCLUSION AND FUTURE WORK

This research concludes with the introduction of a collaborative network monitoring framework that provides users with a unified perspective on the overall infrastructure health while protecting against network-level threats, particularly DDoS attacks. We have effectively demonstrated the capability of the stack through the successful integration and extraction of network metrics from monitoring and visualisation tools such as *Prometheus*, *Zabbix*, and *Grafana*. The deployed

stack demonstrated its proficiency in aggregating vital network health data and metrics, which contributed to the effective detection of less sophisticated DDoS attacks. This capability, coupled with the stack's ability to defend against zero-day exploits, demonstrates the stack's utility for bolstering a company's cybersecurity posture.

The stack can be expanded to include additional modules, such as threat intelligence feeds, application layer monitoring, and organisational identity monitoring, whose useful metrics can be visualised in a single dashboard view to monitor an organisation and strengthen its defences against numerous cyberattacks.

ACKNOWLEDGMENT

This publication has emanated from research supported in part by a Grant from Science Foundation Ireland under Grant number 18/CRT/6222.

REFERENCES

- [1] B. Zhang, T. Zhang, and Z. Yu, "Ddos detection and prevention based on artificial intelligence techniques," in *2017 3rd IEEE International Conference on Computer and Communications (ICCC)*. IEEE, 2017, pp. 1276–1280.
- [2] I. Magazine, "2022: Ddos year-in-review," <https://www.infosecurity-magazine.com/blogs/2022-ddos-yearinreview/>, 2023.
- [3] S. Cook, "20+ ddos attack statistics and facts for 2018-2023," <https://www.comparitech.com/blog/information-security/ddos-statistics-facts/>, 2023.
- [4] S. Lee, K. Levanti, and H. S. Kim, "Network monitoring: Present and future," *Computer Networks*, vol. 65, pp. 84–98, 2014.
- [5] D. C. Wyld, M. Wozniak, N. Chaki, N. Meghanathan, and D. Nagamalai, *Advances in Network Security and Applications: 4th International Conference, CNSA 2011, Chennai, India, July 15-17, 2011, Proceedings*. Springer, 2011, vol. 196.
- [6] J. Mirkovic and P. Reiher, "A taxonomy of ddos attack and ddos defense mechanisms," *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 2, pp. 39–53, 2004.
- [7] P. Faruki, V. Ganmoor, V. Laxmi, M. S. Gaur, and A. Bharmal, "Androsimilar: robust statistical feature signature for android malware detection," in *Proceedings of the 6th International Conference on Security of Information and Networks*, 2013, pp. 152–159.
- [8] P. Faruki, S. Bhandari, V. Laxmi, M. Gaur, and M. Conti, "Droidanalyst: Synergic app framework for static and dynamic app analysis," *Recent Advances in Computational Intelligence in Defense and Security*, pp. 519–552, 2016.
- [9] P. Faruki, A. Bharmal, V. Laxmi, V. Ganmoor, M. S. Gaur, M. Conti, and M. Rajarajan, "Android security: a survey of issues, malware penetration, and defenses," *IEEE communications surveys & tutorials*, vol. 17, no. 2, pp. 998–1022, 2014.
- [10] S. A. R. Shah and B. Issac, "Performance comparison of intrusion detection systems and application of machine learning to snort system," *Future Generation Computer Systems*, vol. 80, pp. 157–170, 2018.
- [11] J. Ye, X. Cheng, J. Zhu, L. Feng, and L. Song, "A ddos attack detection method based on svm in software defined network," *Security and Communication Networks*, vol. 2018, 2018.
- [12] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for iot security based on learning techniques," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2671–2701, 2019.
- [13] Y. Sun, H. Ochiai, and H. Esaki, "Multi-type anomaly detection based on raw network traffic," in *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 2021, pp. 1–2.
- [14] L. Yang and A. Shami, "Ids-ml: An open source code for intrusion detection system development using machine learning," *Software Impacts*, vol. 14, p. 100446, 2022.
- [15] S. Mandala, A. I. Ramadhan, M. Rosalinda, W. M. Yafooz, and R. H. Khohar, "Ddos detection by using information gain-naïve bayes," in *2022 2nd International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*. IEEE, 2022, pp. 283–288.
- [16] S. Kautish, A. Reyana, and A. Vidyarthi, "Sdmta: Attack detection and mitigation mechanism for ddos vulnerabilities in hybrid cloud environment," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 9, pp. 6455–6463, 2022.
- [17] B. Mohammed, M. Kiran, and B. Enders, "Netgraf: An end-to-end learning network monitoring service," in *2021 IEEE Workshop on Innovating the Network for Data-Intensive Science (INDIS)*. IEEE, 2021, pp. 12–22.
- [18] Z. Maasaoui, A. Hathah, H. Bilil, V. S. Mai, A. Battou, and A. Lbath, "Network security traffic analysis platform-design and validation," in *2022 IEEE/ACS 19th International Conference on Computer Systems and Applications (AICCSA)*. IEEE, 2022, pp. 1–5.
- [19] R. Doshi, N. Aphorpe, and N. Feamster, "Machine learning ddos detection for consumer internet of things devices," in *2018 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2018, pp. 29–35.
- [20] GNS3, "Gns3 software download," <https://www.gns3.com/software/download>, 2023.

APPENDIX

A. GNS3 Installation and Configuration

- Step 1. Download GNS3 from the GNS3 website [20].
- Step 2. Install the software alongside its default dependencies to get started. Examine the list of dependencies to guarantee the GNS3 VM is included as part of this procedure. If the GNS3 virtual machine is listed, you can choose to deploy it during installation. This phase is required for the installation and operation of customised routers and switches that may not be pre-loaded with GNS3.
- Step 3. Once installed, establish a new Blank project to construct the network design.
- Step 4. Access the devices tab to integrate the necessary devices for network design.
- Step 5. Design and configure the network in accordance with the intended network design.

B. Zabbix Installation and Configuration

Zabbix is installed in a Centos VM having 2 GB RAM and storage of 20 GB.

- Step 1. Download Zabbix version 5.0 from by using the command shown in Fig. 6. This command will install the Zabbix repo on the Centos machine.
- Step 2. Install Zabbix server, Zabbix front end, and Zabbix agent.
- Step 3. Install Zabbix front end packages.
- Step 4. Install Maria db to run mysql database for Zabbix.
- Step 5. Login into mysql console and install a database for Zabbix.
- Step 7. Start the Zabbix services (Zabbix server, Zabbix agent, apache webserver and Zabbix front end)
- Step 8. Enable the Zabbix services so they always start when the machine startup.
- Step 9. Open the browser and type in the following url to access Zabbix front end.
- Step 10. Login with the username Admin and password Zabbix.
- Step 11. Right now Zabbix server is only monitoring itself through a Zabbix agent. To add additional devices, select hosts from the configuration tab on the left panel.

```

Step 1: rpm -Uvh %https://repo.zabbix.com/zabbix/5.0/rhel/7/x86_64/zabbix-release-5.0-1.el7.noarch.rpm
Step 2: yum install zabbix-server-mysql zabbix-agent
Step 3: yum install zabbix-web-mysql-scl zabbix-apache-conf-scl
Step 4: sudo yum install mariadb-server

Step 5: # mysql -uroot -p
<mysql_password>
mysql> create database zabbix character set utf8 collate utf8_bin;
mysql> create user zabbix@localhost identified by '<password>';
mysql> grant all privileges on zabbix.* to zabbix@localhost;
mysql> set global log_bin_trust_function_creators = 1;
mysql> quit;

Step 6: zcat /usr/share/doc/zabbix-server-mysql*/create.sql.gz |
mysql -uzabbix -p <zabbix_password>

Step 7: systemctl start zabbix-server zabbix-agent httpd rh-php72-php-fpm

Step 8: http://<zabbix_IP_Address>/zabbix

```

Fig. 6. Zabbix Installation and Configuration Steps Commands

```

Step 1: #wget http://repo.zabbix.com/zabbix/3.4/ubuntu/pool/main/z/zabbix-release/zabbix-release_3.4-1+xenial_all.deb
Step 2: #sudo apt-get install zabbix-agent
Step 4: #sudo vi /etc/zabbix/zabbix_agentd.conf

```

Fig. 7. Zabbix Agent Installation and Configuration Commands

- Step 12. Click on "Create Host" button.
- Step 13. To add an Ubuntu node, specify its hostname, IP address, and Linux-based template.
- Step 14. Click the "Add" icon. The Ubuntu node is now accessible for monitoring. On the node side, however, the *Zabbix* agent must be installed and configured. Refer to section 5.3 for installation instructions.

C. Zabbix Agent Installation and Configuration

- Step 1. Run the following command on the ubuntu terminal to add *Zabbix* agent repository.
- Step 2. Execute the command below in the Ubuntu terminal to install the *Zabbix* agent as shown in Fig 7.
- Step 3. Open the *Zabbix* agent configuration file as shown in Fig 7.
- Step 4. Under the Server section of the configuration file enter the IP address of the *Zabbix* server.

```

### Option: Server
# List of comma delimited IP addresses, optionally in CIDR notation, or DNS names of Zabbix
# Incoming connections will be accepted only from the hosts listed here.
# If IPv6 support is enabled then '127.0.0.1', '::127.0.0.1', '::ffff:127.0.0.1' are treated
# and ':::0' will allow any IPv4 or IPv6 address.
# '0.0.0.0/0' can be used to allow any IPv4 address.
# Example: Server=127.0.0.1,192.168.1.0/24,::1,2001:db8::32,zabbix.example.com
#
# Mandatory: yes, if StartAgents is not explicitly set to 0
# Default:
# Server=
Server=<zabbix server IP Address>

```

Fig. 8. Zabbix Agent config file

- Step 5. Restart the agent service for *Zabbix*. Now the *Zabbix* server will be able to monitor the node and receive metrics.

```
Step 5: #sudo systemctl restart zabbix-agent
```

Fig. 9. Command to restart Zabbix Agent

```

Step 1: #wget https://github.com/prometheus/prometheus/releases/download/v2.37.6/prometheus-2.37.6.linux-amd64.tar.gz
Step 2: #tar xvfz prometheus-*.tar.gz
Step 3: #sudo mkdir /etc/prometheus /var/lib/prometheus
Step 4: #sudo mv prometheus promtool /usr/local/bin/
Step 5: #sudo mv Prometheus.yml /etc/prometheus/prometheus.yml
Step 6: #sudo systemctl start prometheus
Step 7: http://<prometheus_IP_Address>:9090

```

Fig. 10. Prometheus Installation and Configuration Steps Commands

D. Prometheus Installation and Configuration

- Step 1. Download *Prometheus* by using the commands in Fig 10.
- Step 2. Extract the downloaded files
- Step 3. Create the 2 directories for *Prometheus* to use and store it's configuration files using the following command.
- Step 4. Move the directories *Prometheus* and prom tool to the *usr/local/bin* directory.
- Step 5. Move the *Prometheus.yml* file to the *etc/prometheus* directory.
- Step 6. Run the following command to start *Prometheus*.
- Step 7. Open a browser and enter the following url to log into the *Prometheus* console.

E. Grafana Installation and Configuration

```

Step 1: #sudo apt-get install grafana
Step 2: #systemctl start grafana-server
#systemctl enable grafana-server
Step 3: http://<Grafana-Server-IP-Address>:3000

```

Fig. 11. Grafana Installation and Configuration Steps Commands

- Step 1. Run the commands shown in Fig 11 on the ubuntu terminal to install *Grafana*.
- Step 2. Start the *Grafana* server and enable it
- Step 3. To access the *Grafana* front end, open a web browser and enter the following URL. Admin is the default username, and password is the default password.
- Step 4. Once Logged in on the left panel click on Configuration. and select data source to add data sources.
- Step 5. Click the data source connector you wish to add and enter details such as the data source's URL and authentication information. Figure 9 is a screenshot of '*Prometheus*' being added as a data source to '*Grafana*'.
- Step 6. Once the data sources have been added successfully, navigate to the left panel and select Dashboard to create a new dashboard.

- Step 7. Select the option to add a new panel in the upper-right corner. This link will navigate you to the *Grafana* Panel editor. Here, configurations can be made based on the statistic that needs to be displayed or visualised. and in the query editor For the purposes of the Research, only time-series and graphical representation of statistics were employed.
- Step 8. Navigate to the right panel and locate the threshold bar. In the panel, it is possible to establish a manual threshold that is tailored to the specific characteristics of the data being visualised.
- Step 9. After visualising the desired metric in the proper format, proceed to click on the "apply" button.
- Step 10. Click save on the Dashboard panel to save the new dashboard.